```
project-folder
│
├── server.js
├── database.json
└── public
    ├── login.html
    ├── register.html
    ├── profile.html
    └── style.css
```

This version will:

- ✅ Register users
- ✅ Login users
- ✅ Generate JWT
- ✅ Validate with middleware
- ✅ Redirect to profile
- ✅ Store token in localStorage
- ✅ Protect profile page

📁 database.json

```
{
  "users": [
    {
      "id": 1,
      "username": "admin",
      "password": "1234"
    },
    {
      "id": 2,
      "username": "john",
      "password": "pass123"
    }
  ]
}
```

📁 server.js

```
const express = require("express");
const jwt = require("jsonwebtoken");
const fs = require("fs");
const path = require("path");

const app = express();
const PORT = 3000;
const SECRET_KEY = "supersecretkey";

app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

const dbPath = path.join(__dirname, "database.json");

// Read database
function readDatabase() {
  return JSON.parse(fs.readFileSync(dbPath, "utf-8"));
}

// Write database
function writeDatabase(data) {
  fs.writeFileSync(dbPath, JSON.stringify(data, null, 2));
}

/* ==============================
   JWT AUTH MIDDLEWARE
================================== */
function authenticateToken(req, res, next) {
  const authHeader = req.headers["authorization"];
  const token = authHeader && authHeader.split(" ")[1];

  if (!token) return res.status(401).json({ message: "Token required" });

  jwt.verify(token, SECRET_KEY, (err, user) => {
    if (err) return res.status(403).json({ message: "Invalid token" });

    req.user = user;
    next();
  });
}

/* ==============================
```

```
  REGISTER
================================== */
app.post("/register", (req, res) => {
  const { username, password } = req.body;

  const db = readDatabase();

  const userExists = db.users.find(u => u.username === username);
  if (userExists) {
    return res.status(400).json({ message: "User already exists" });
  }

  const newUser = {
    id: Date.now(),
    username,
    password
  };

  db.users.push(newUser);
  writeDatabase(db);

  res.json({ message: "Registration successful" });
});

/* ==============================
  LOGIN
================================== */
app.post("/login", (req, res) => {
  const { username, password } = req.body;

  const db = readDatabase();

  const user = db.users.find(
    u => u.username === username && u.password === password
  );

  if (!user) {
    return res.status(401).json({ message: "Invalid credentials" });
  }

  const token = jwt.sign(
    { id: user.id, username: user.username },
    SECRET_KEY,
    { expiresIn: "1h" }
```

```
  );

  res.json({ token });
console.log('The generated token is:', token);
});

/* ==============================
   PROFILE (Protected)
================================== */
app.get("/profile-data", authenticateToken, (req, res) => {
  res.json({
    message: "Welcome to your profile",
    user: req.user
  });
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

📁 public/login.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h2>Login</h2>
    <form id="loginForm">
      <input type="text" id="username" placeholder="Username" required>
      <input type="password" id="password" placeholder="Password" required>
      <button type="submit">Login</button>
    </form>
    <p>Don't have an account? <a href="register.html">Register</a></p>
  </div>

<script>
document.getElementById("loginForm").addEventListener("submit", async (e) => {
```

```
  e.preventDefault();

  const username = document.getElementById("username").value;
  const password = document.getElementById("password").value;

  const res = await fetch("/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ username, password })
  });

  const data = await res.json();

  if (res.ok) {
    localStorage.setItem("token", data.token);
    window.location.href = "profile.html";
  } else {
    alert(data.message);
  }
});
</script>
</body>
</html>
```

📁 public/register.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Register</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h2>Register</h2>
    <form id="registerForm">
      <input type="text" id="username" placeholder="Username" required>
      <input type="password" id="password" placeholder="Password" required>
      <button type="submit">Register</button>
    </form>
    <p>Already have an account? <a href="login.html">Login</a></p>
```

```
  </div>

<script>
document.getElementById("registerForm").addEventListener("submit", async (e) => {
  e.preventDefault();

  const username = document.getElementById("username").value;
  const password = document.getElementById("password").value;

  const res = await fetch("/register", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ username, password })
  });

  const data = await res.json();

  if (res.ok) {
    alert("Registration successful! Please login.");
    window.location.href = "login.html";
  } else {
    alert(data.message);
  }
});
</script>
</body>
</html>
```

📁 public/profile.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Profile</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h2>Profile</h2>
    <div id="profileData"></div>
    <button onclick="logout()">Logout</button>
```

```
  </div>

<script>
async function loadProfile() {
  const token = localStorage.getItem("token");

  if (!token) {
    window.location.href = "login.html";
    return;
  }

  const res = await fetch("/profile-data", {
    headers: {
      "Authorization": "Bearer " + token
    }
  });

  if (!res.ok) {
    alert("Session expired. Please login again.");
    localStorage.removeItem("token");
    window.location.href = "login.html";
    return;
  }

  const data = await res.json();
  document.getElementById("profileData").innerHTML =
    `<p><strong>Username:</strong> ${data.user.username}</p>`;
}

function logout() {
  localStorage.removeItem("token");
  window.location.href = "login.html";
}

loadProfile();
</script>
</body>
</html>
```

📁 public/style.css

```
body {
  font-family: Arial, sans-serif;
  background: #f4f4f4;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}

.container {
  background: white;
  padding: 30px;
  border-radius: 10px;
  width: 300px;
  text-align: center;
  box-shadow: 0 0 10px rgba(0,0,0,0.1);
}

input {
  width: 100%;
  padding: 10px;
  margin: 10px 0;
}

button {
  width: 100%;
  padding: 10px;
  background: #4CAF50;
  color: white;
  border: none;
  cursor: pointer;
}

button:hover {
  background: #45a049;
}

a {
  text-decoration: none;
  color: #4CAF50;
}
```

🚀 How to Run

npm install express jsonwebtoken
node server.js

Then open:

http://localhost:3000/login.html

Yes ✅ — you can absolutely run your project **without Postman**.

Since you already built:

- `login.html`
- `register.html`
- `profile.html`

You can test everything directly in your **browser**.

🔎 What is Postman?

Postman is an API testing tool used to:

- Send HTTP requests (GET, POST, PUT, DELETE)
- Test backend APIs without a frontend
- Send JSON body manually
- Add headers like Authorization
- Debug API responses

It is mainly used by:

- Backend developers
- API testers
- QA engineers

🖥️ Can You Run Without Postman?

✅ YES — in your case you **don't need Postman** because:

- Your frontend (HTML + JS) already sends requests using `fetch()`
- The browser handles everything
- JWT is stored in localStorage
- Profile page sends Authorization header automatically

Just run:

node server.js

Then open:

http://localhost:3000/login.html

That's it 🚀

🤔 When Should You Use Postman?

Use Postman when:

- You are building **only backend**
- You don't have frontend yet
- You want to test routes quickly
- You want to debug errors
- You want to check response status codes

Example:
Testing `/login` API without HTML.

🆚 Which Is Better?

✅ For Your Project (College / Learning JWT)

👉 **Without Postman (Browser method) is better**
Because:

- It shows full working system
- Demonstrates login → token → profile
- Looks more complete in viva

✅ For Backend Developer Practice

👉 Postman is better
Because:

- Faster debugging
- Clear API testing
- Industry standard tool

🏆 Best Recommendation

For your current setup:

✔ Run using browser
✔ Keep Postman only for extra testing

🎓 Simple Comparison

With Browser	With Postman
Full project demo	API only
Real user experience	Backend testing
Good for presentation	Good for debugging
Easier	More technical

💡 Viva Tip

If examiner asks:

"Why are you not using Postman?"

You can say:

> "Since I built a frontend interface, I tested the APIs directly through the browser. Postman is mainly used when testing APIs without a frontend."

That sounds professional 👍

JWT in your project step-by-step, exactly according to your code.

We'll follow what happens from **Login** → **Token Creation** → **Token Storage** → **Protected Route Access** → **Verification**.

🔐 STEP 1: User Sends Login Request

From **login.html**, this runs:

```
const res = await fetch("/login", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ username, password })
});
```

📌 What happens here?

- Browser sends:
  - URL → `/login`
  - Method → POST
  - Body → username & password
- This request goes to your **server.js**

🔐 STEP 2: Server Validates User

In `server.js`:

```
app.post("/login", (req, res) => {
  const { username, password } = req.body;

  const db = readDatabase();

  const user = db.users.find(
    u => u.username === username && u.password === password
  );
```

📌 What happens?

1. Server reads database.json
2. Checks if username & password match
3. If no match → return 401 (Invalid credentials)
4. If match → move to JWT creation

🔐 STEP 3: JWT Is Created

```
const token = jwt.sign(
  { id: user.id, username: user.username },
  SECRET_KEY,
  { expiresIn: "1h" }
);
```

This is the MOST IMPORTANT LINE.

What `jwt.sign()` does:

It creates a token with:

1️⃣ Payload

{ id: user.id, username: user.username }

This is the user information stored inside token.

2️⃣ Secret Key

SECRET_KEY = "supersecretkey"

This key:

- Signs the token
- Prevents tampering
- Must be kept secret

3️⃣ Expiration

{ expiresIn: "1h" }

Token will expire after 1 hour.

🧱 What Does JWT Actually Look Like?

A JWT looks like:

xxxxx.yyyyy.zzzzz

It has 3 parts:

🔹 1. Header

Contains algorithm type:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

🔹 2. Payload

Contains:

```
{
  "id": 2,
  "username": "john",
  "exp": 1712345678
}
```

🔹 3. Signature

Created using:

Header + Payload + SECRET_KEY

If someone changes payload → signature becomes invalid.

🔐 STEP 4: Token Sent to Browser

```
res.json({ token });
```

Browser receives:

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

🔐 STEP 5: Token Stored in localStorage

In login.html:

```
localStorage.setItem("token", data.token);
```

Now browser stores token locally.

So user is considered “logged in”.

🔐 STEP 6: Accessing Protected Page

When profile.html loads:

```
const token = localStorage.getItem("token");
```

Then it sends request:

```
fetch("/profile-data", {
  headers: {
    "Authorization": "Bearer " + token
  }
});
```

📌 Important:
The token is sent in HTTP Header:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

🔐 STEP 7: Middleware Checks Token

Now this runs:

```
function authenticateToken(req, res, next) {
  const authHeader = req.headers["authorization"];
  const token = authHeader && authHeader.split(" ")[1];

  if (!token) return res.status(401).json({ message: "Token required" });

  jwt.verify(token, SECRET_KEY, (err, user) => {
    if (err) return res.status(403).json({ message: "Invalid token" });
```

```
    req.user = user;
    next();
  });
}
```

Let’s break it down:

🔹 1️⃣ Extract Token

```
const token = authHeader.split(" ")[1];
```

Removes "Bearer "
Gets only the token part.

🔹 2️⃣ If No Token

Return:

401 Unauthorized

🔹 3️⃣ Verify Token

```
jwt.verify(token, SECRET_KEY)
```

This checks:

✔ Is signature valid?
✔ Was token modified?
✔ Is token expired?

If anything fails → 403 error.

🔹 4️⃣ If Valid

```
req.user = user;
next();
```

Now user info becomes available in route.

🔐 STEP 8: Protected Route Executes

```
app.get("/profile-data", authenticateToken, (req, res) => {
  res.json({
    message: "Welcome to your profile",
    user: req.user
  });
});
```

Since middleware passed:

You now receive:

```
{
  "message": "Welcome to your profile",
  "user": {
    "id": 2,
    "username": "john",
    "iat": 123456,
    "exp": 123456
  }
}
```

🧠 What Makes JWT Special?

JWT is:

✔ Stateless
✔ No session storage needed
✔ No server memory required
✔ Scalable
✔ Fast

Server does NOT store sessions.
Token itself contains user info.

🆚 JWT vs Traditional Session

Traditional:

- Server stores session in memory
- Client stores session ID
- Needs database or memory

JWT:

- No server storage
- Token contains data
- Easy for distributed systems

🎓 If Examiner Asks: “Explain JWT Flow”

You can say:

> When user logs in, the server validates credentials and generates a signed JWT containing user information. The token is sent to the client and stored in localStorage. For protected routes, the client sends the token in the Authorization header. The server verifies the token using the secret key. If valid, access is granted; otherwise, access is denied.

That is a perfect answer 👍

🔍1️⃣See Token in Browser (Easiest Method)

After login:

```
localStorage.setItem("token", data.token);
```

So the token is stored in **localStorage**.

👉 To View It:

1. Open your app in browser
2. Press **F12** (Developer Tools)
3. Go to **Application** tab
4. Click **Local Storage**
5. Click `http://localhost:3000`
6. You will see:

Key: token

Value: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

🎉 That long string is your JWT.

🔍2️⃣ See Token in Network Request

You can also see it being sent:

1. Open **F12**
2. Go to **Network** tab
3. Click `profile-data`
4. Go to **Headers**
5. Scroll to:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

That shows the token being sent to the server.

🔍3️⃣ Decode and Visualize Token Content

JWT has 3 parts:

HEADER.PAYLOAD.SIGNATURE

Copy your token from localStorage.

Then go to:

👉 https://jwt.io

Paste the token in the left box.

You will see:

🔹 Header


```
{
  "alg": "HS256",
  "typ": "JWT"
}
```


🔹 Payload


```
{
  "id": 2,
  "username": "john",
  "iat": 1712345678,
  "exp": 1712349278
}
```


Now you can visually see:

- id
- username
- issued at time

- expiration time

⚠️ Important:
This does NOT show your secret key.
Signature cannot be decoded without secret.

🧠 Important Understanding

JWT is:

- Base64 encoded
- NOT encrypted

Anyone can decode the payload.
That is why:

❌ Never store password inside JWT
❌ Never store sensitive data

Only store:

- id
- username
- role

🧪4️⃣ You Can Also Log It in Server

If you want to see token when generated:

Add this in `/login`:

console.log("Generated Token:", token);

When you login, terminal will show:

Generated Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

🎓 If Examiner Asks:

“Can you see JWT?”

You can say:

> Yes, JWT is visible in the browser's localStorage and in the Authorization header. It can also be decoded using tools like jwt.io to inspect the header and payload.

That’s a strong answer 👍

⚡ Extra: Want to Display Token on Profile Page?

You could temporarily add:

```
document.getElementById("profileData").innerHTML +=

`<p><strong>Token:</strong> ${token}</p>`;
```

(Not recommended in real apps, but good for demo.)

🔥 Final Answer

Yes — you can:

✔ View token in localStorage
✔ See it in Network tab
✔ Decode it visually
✔ Print it in server console